VNRVJIET

NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:

WEEK-1

Title of the Experiment: Self-Driving Car using Deep Learning

Objective of the experiment:

To develop a self-driving car model using deep learning techniques by leveraging a convolutional neural network (CNN) for steering angle prediction. Train and evaluate the model on real work self-driving datasets.

Implementation Steps and Code:

a. **Problem Definition and Dataset**
   - Understand the problem of autonomous driving.
   - Use the Udacity self-driving car dataset (or any open-source dataset)

b. **Data understandings analysis files and folders**

```
!pip install -q tensorflow opencv-python numpy pandas matplotlib scikit-learn

import os
import random
import ntpath

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.image as mpimg


import cv2

from sklearn.utils import shuffle
from sklearn.model_selection import train_test_split

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, Dense, MaxPooling2D, Dropout, Flatten
from tensorflow.keras.optimizers import Adam

!git clone https://github.com/Aman-py/data.git

columns = ['center','left','right','steering','throttle','reverse','speed']
```

```
datadir = 'data'
dataset = pd.read_csv(
    os.path.join(datadir, 'driving_log.csv'),
    names=columns
)

dataset.head()
```

	center	left	right	steering	throttle	reverse	speed
0	/home/aman-py/Desktop/Self-Driving-Car/data/IM...	/home/aman-py/Desktop/Self-Driving-Car/data/I...	/home/aman-py/Desktop/Self-Driving-Car/data/I...	0.0	0.0	0.0	0.000078
1	/home/aman-py/Desktop/Self-Driving-Car/data/IM...	/home/aman-py/Desktop/Self-Driving-Car/data/I...	/home/aman-py/Desktop/Self-Driving-Car/data/I...	0.0	0.0	0.0	0.000078
2	/home/aman-py/Desktop/Self-Driving-Car/data/IM...	/home/aman-py/Desktop/Self-Driving-Car/data/I...	/home/aman-py/Desktop/Self-Driving-Car/data/I...	0.0	0.0	0.0	0.000079
3	/home/aman-py/Desktop/Self-Driving-Car/data/IM...	/home/aman-py/Desktop/Self-Driving-Car/data/I...	/home/aman-py/Desktop/Self-Driving-Car/data/I...	0.0	0.0	0.0	0.000078
4	/home/aman-py/Desktop/Self-Driving-Car/data/IM...	/home/aman-py/Desktop/Self-Driving-Car/data/I...	/home/aman-py/Desktop/Self-Driving-Car/data/I...	0.0	0.0	0.0	0.000079

```
print("Number of samples:", len(dataset))
print("\nColumns:")
print(dataset.columns)

Number of samples: 7259

Columns:
Index(['center', 'left', 'right', 'steering', 'throttle', 'reverse', 'speed'], dtype='object')

#Clean image paths
def removePath(path):
    base, tail = ntpath.split(path)
    return tail
```

WEEK - 2

c. Dash-com, images and steering analysis

```
dataset['center'] = dataset['center'].apply(removePath)
dataset['left'] = dataset['left'].apply(removePath)
dataset['right'] = dataset['right'].apply(removePath)
```

```
dataset.head()
```

	center	left	right	steering	throttle	reverse	speed
0	center_2018_10_17_16_36_30_865.jpg	left_2018_10_17_16_36_30_865.jpg	right_2018_10_17_16_36_30_865.jpg	0.0	0.0	0.0	0.000078
1	center_2018_10_17_16_36_30_966.jpg	left_2018_10_17_16_36_30_966.jpg	right_2018_10_17_16_36_30_966.jpg	0.0	0.0	0.0	0.000078
2	center_2018_10_17_16_36_31_098.jpg	left_2018_10_17_16_36_31_098.jpg	right_2018_10_17_16_36_31_098.jpg	0.0	0.0	0.0	0.000079
3	center_2018_10_17_16_36_31_253.jpg	left_2018_10_17_16_36_31_253.jpg	right_2018_10_17_16_36_31_253.jpg	0.0	0.0	0.0	0.000078
4	center_2018_10_17_16_36_31_366.jpg	left_2018_10_17_16_36_31_366.jpg	right_2018_10_17_16_36_31_366.jpg	0.0	0.0	0.0	0.000079

```
#Analyze steering angles

num_bins = 25
hist, bins = np.histogram(dataset['steering'],num_bins)
print("Histogram:")
print(hist)
print("\nBin edges:")
print(bins)
```

```
Histogram:
[ 48   16   13   15   21   36   48   69  161  174  337  239 5454   56
  160   82   97   99   37   25   32   15   12    5    8]

Bin edges:
[-1.   -0.92 -0.84 -0.76 -0.68 -0.6  -0.52 -0.44 -0.36 -0.28 -0.2  -0.12
 -0.04  0.04  0.12  0.2   0.28  0.36  0.44  0.52  0.6   0.68  0.76  0.84
  0.92  1.  ]
```

```
#Calculate bin centers
center = (bins[:-1] + bins[1:]) * 0.5
print(center)
```

```
[-0.96 -0.88 -0.8  -0.72 -0.64 -0.56 -0.48 -0.4  -0.32 -0.24 -0.16 -0.08
  0.    0.08  0.16  0.24  0.32  0.4   0.48  0.56  0.64  0.72  0.8   0.88
  0.96]
```

```
#Plot steering-angle distribution
threshold = 500

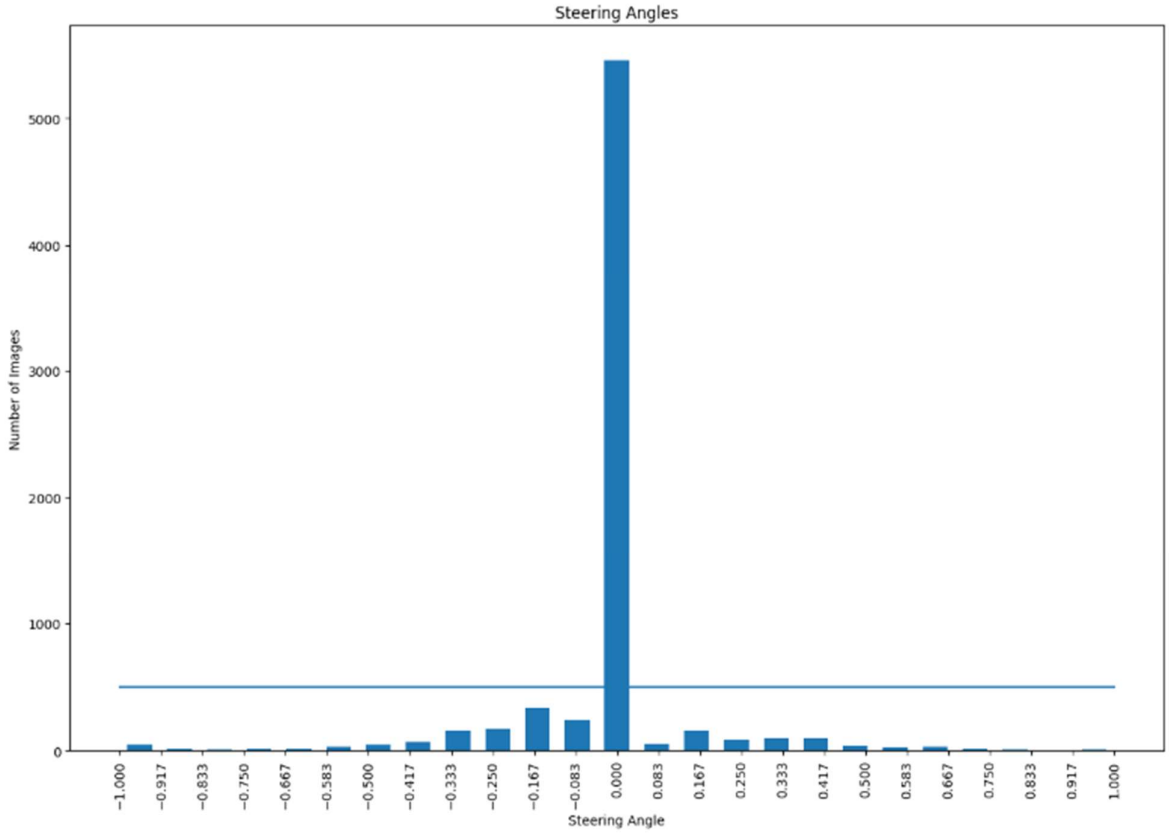
plt.figure(figsize=(15, 10))
```

```
plt.bar(center,hist,width=0.05)
plt.xticks(
    np.linspace(-1, 1, 25),
    rotation=90
)

plt.title("Steering Angles")
plt.xlabel("Steering Angle")
plt.ylabel("Number of Images")

x1 = np.min(dataset['steering'])
x2 = np.max(dataset['steering'])
plt.plot( (x1, x2),(threshold, threshold))

plt.show()
```




```
#Remove excessive steering-angle samples

remove_list = []
```

```
for i in range(num_bins):

    bin_indices = []

    for j in range(len(dataset['steering'])):

        if (dataset['steering'].iloc[j] >= bins[i] and dataset['steering'].iloc[j] <= bins[i + 1]):
            bin_indices.append(j)

    bin_indices = shuffle(bin_indices, random_state=42)

    # Remove samples above threshold
    if len(bin_indices) > threshold:
        bin_indices = bin_indices[threshold:]

    remove_list.extend(bin_indices)

print("Samples to remove:", len(remove_list))
```

Samples to remove: 6759

```
dataset = dataset.drop(dataset.index[remove_list])
dataset = dataset.reset_index(drop=True)
print("Dataset size after balancing:", len(dataset))
```

Dataset size after balancing: 500

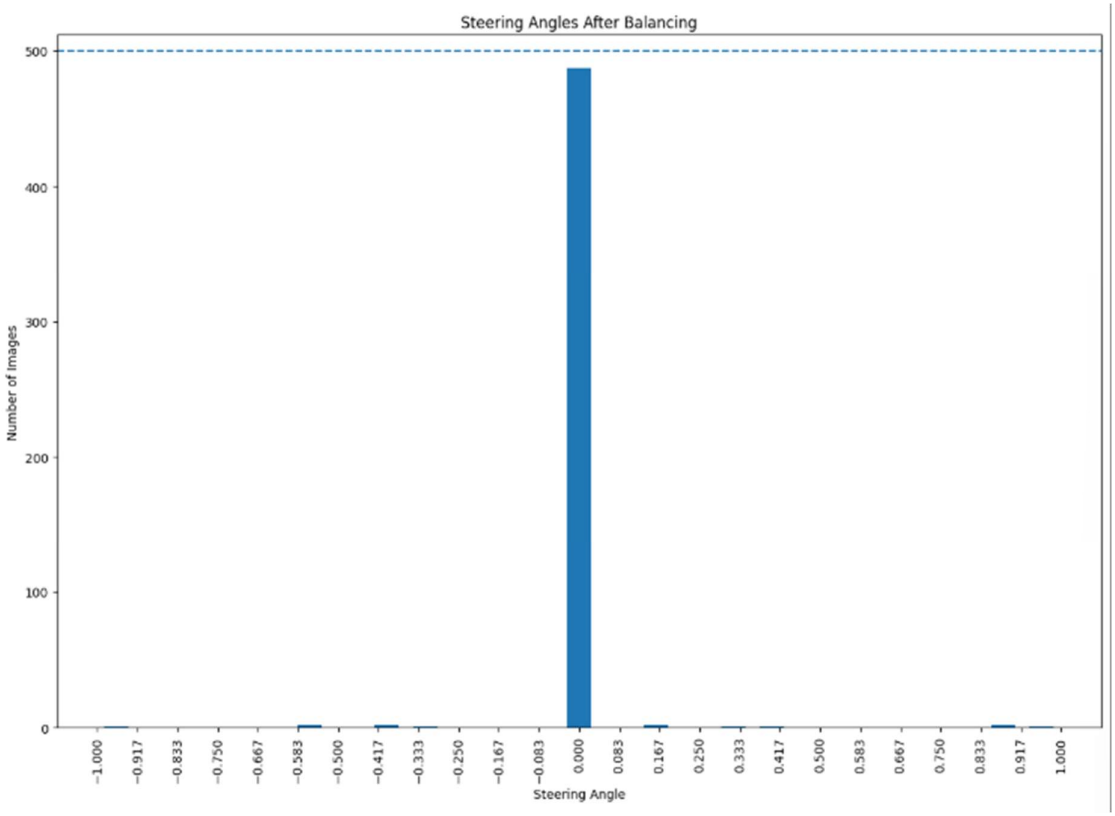
```
#Check the balanced steering distribution

hist, _ = np.histogram(dataset['steering'],num_bins)

plt.figure(figsize=(15, 10))
plt.bar(center,hist,width=0.05)
plt.xticks(np.linspace(-1, 1, 25), rotation=90)

plt.title("Steering Angles After Balancing")
plt.xlabel("Steering Angle")
plt.ylabel("Number of Images")

plt.axhline(threshold, linestyle='--')
plt.show()
```



dataset.iloc[1]

	1
center	center_2018_10_17_16_36_32_664.jpg
left	left_2018_10_17_16_36_32_664.jpg
right	right_2018_10_17_16_36_32_664.jpg
steering	0.0
throttle	0.0
reverse	0.0
speed	0.000078

dtype: object

```
#Load image paths and steering values
def loadImageSteering(datadir, dataset):

    imagePath = []
    steeringPath = []

    for i in range(len(dataset)):
```

```
        center_image = dataset.iloc[i]['center']
        steering = float(dataset.iloc[i]['steering'])
        imagePath.append(os.path.join(datadir, center_image))
        steeringPath.append(steering)

    imagePath = np.asarray(imagePath)
    steeringPath = np.asarray(steeringPath)

    return imagePath, steeringPath

imagePath, steeringPath = loadImageSteering(os.path.join(datadir, 'IMG'),dataset)

print("First image:")
print(imagePath[0])

print("\nNumber of images:", len(imagePath))
print("Number of steering values:", len(steeringPath))
print(os.path.exists(imagePath[0]))
```

```
First image:
data/IMG/center_2018_10_17_16_36_31_862.jpg

Number of images: 500
Number of steering values: 500
True
```

d. EDA steering angles

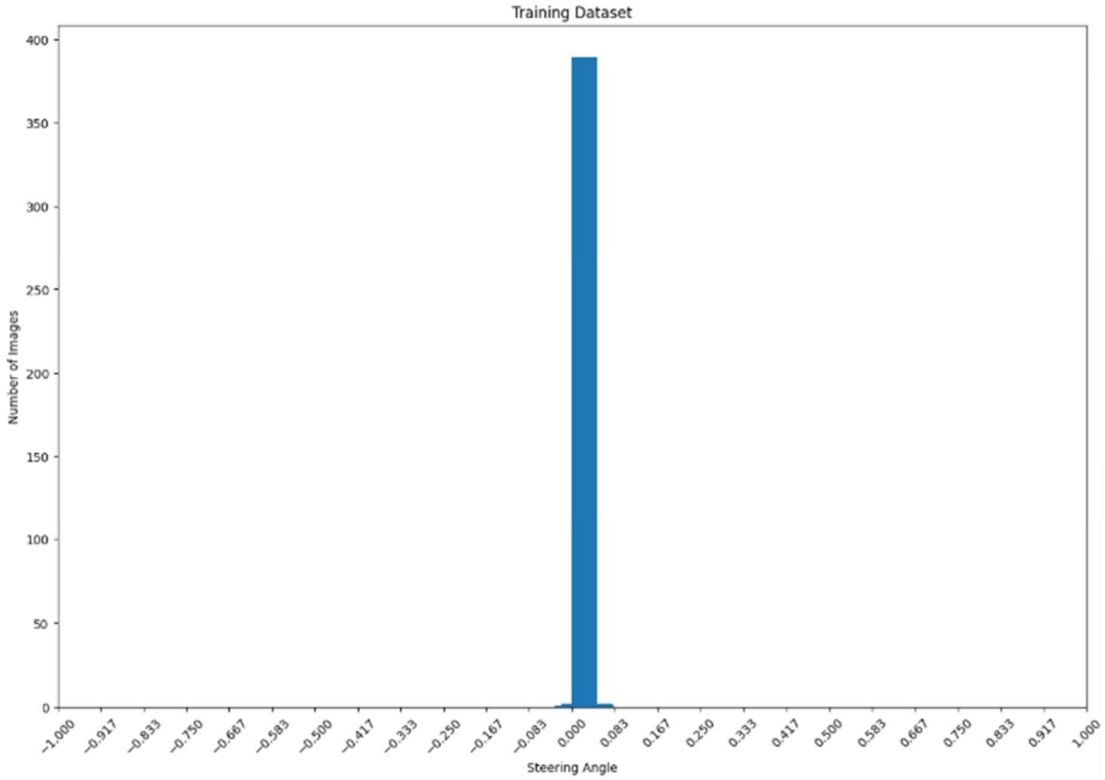
```
#Train-test split
x_train, x_test, y_train, y_test = train_test_split(
    imagePath,
    steeringPath,
    test_size=0.2,
    random_state=6
)

print("Training samples:", len(x_train))
print("Testing samples:", len(x_test))
```

```
Training samples: 400
Testing samples: 100
```

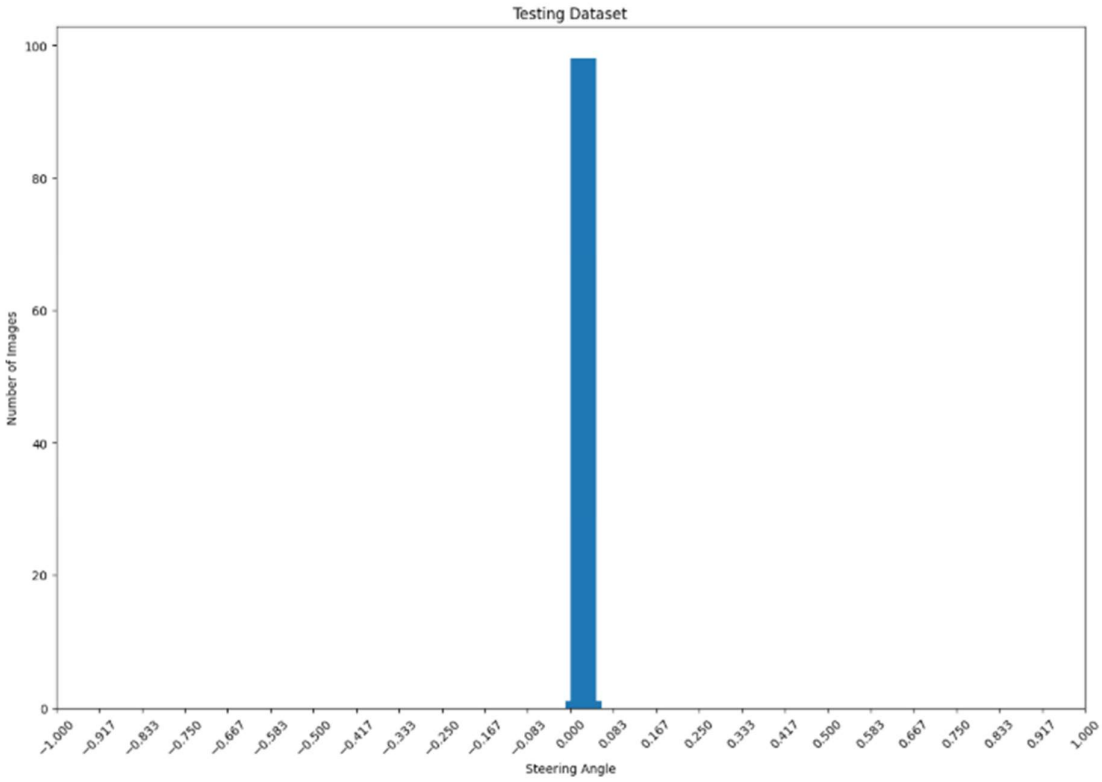
```
plt.figure(figsize=(15, 10))
plt.hist(y_train, bins=num_bins, width=0.05)
plt.xticks(np.linspace(-1, 1, 25), rotation=45)
plt.title("Training Dataset")
plt.xlabel("Steering Angle")
plt.ylabel("Number of Images")

plt.show()
```




```
plt.figure(figsize=(15, 10))
plt.hist(y_test, bins=num_bins, width=0.05)
plt.xticks(np.linspace(-1, 1, 25), rotation=45)
plt.title("Testing Dataset")
plt.xlabel("Steering Angle")
plt.ylabel("Number of Images")

plt.show()
```



e. Mean baseline model

```
mean_steering = np.mean(y_train)
baseline_predictions = np.full(len(y_test), mean_steering)
baseline_mse = np.mean((y_test - baseline_predictions) ** 2)

print("Mean steering angle:", mean_steering)
print("Baseline MSE:", baseline_mse)
```

Mean steering angle: 5.4897042500000006e-05
Baseline MSE: 2.1800227511161654e-06

WEEK - 3

f. Deep learning model, Deep learning for regression

```
def imagePreprocessing(img_path):

    # Read image
    img = mpimg.imread(img_path)

    # Crop the image
    # Remove sky and car hood
    img = img[60:135, :, :]

    # Convert RGB to YUV
   img = cv2.cvtColor(img, cv2.COLOR_RGB2YUV)

    # Apply Gaussian blur
    img = cv2.GaussianBlur(img, (3, 3), 0)

    # Resize to NVIDIA model input size
    img = cv2.resize(img, (200, 66))

    # Normalize pixel values
    img = img / 255.0

    return img

sample_image = x_train[0]

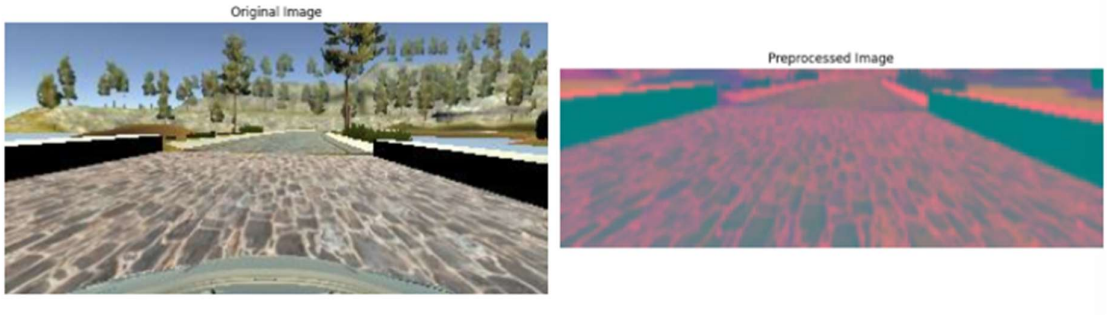
original = mpimg.imread(sample_image)
processed = imagePreprocessing(sample_image)

fig, axs = plt.subplots(1, 2, figsize=(15, 6))

axs[0].imshow(original)
axs[0].set_title("Original Image")
axs[0].axis("off")

axs[1].imshow(processed)
axs[1].set_title("Preprocessed Image")
axs[1].axis("off")

plt.tight_layout()
plt.show()
```




```
sample = imagePreprocessing(x_train[0])
print("Image shape:", sample.shape)
print("Steering angle:", y_train[0])
```

```
Image shape: (66, 200, 3)
Steering angle: 0.0
```

g. Batch load the dataset

```
def batch_generator(image_paths, steering_values, batch_size=32):
    while True:
        # Shuffle the dataset every epoch
        indices = np.random.permutation(len(image_paths))

        for start in range(0, len(image_paths), batch_size):
            # Select indices for this batch
            batch_indices = indices[start:start + batch_size]
            images = []
            steering_angles = []

            for index in batch_indices:
                # Load and preprocess image
                image = imagePreprocessing(image_paths[index])
                images.append(image)
                steering_angles.append(steering_values[index])

            # Convert to NumPy arrays
            X = np.array(images, dtype=np.float32)
            Y = np.array(steering_angles, dtype=np.float32)

            yield X, Y
batch_size = 32
```

```
train_generator = batch_generator(x_train, y_train,batch_size)
validation_generator = batch_generator(x_test,y_test,batch_size)
X_batch, Y_batch = next(train_generator)

print("Batch image shape:", X_batch.shape)
print("Batch steering shape:", Y_batch.shape)
```

```
Batch image shape: (32, 66, 200, 3)
Batch steering shape: (32,)
```

h. NVIDA's end to end CNN MODEL

```
def nvidiaModel():
    model = Sequential()
    # Layer 1
    model.add(Conv2D(24,(5, 5),strides=(2, 2),input_shape=(66, 200, 3),activation='elu'))
    # Layer 2
    model.add(Conv2D(36,(5, 5),strides=(2, 2),activation='elu'))
    # Layer 3
    model.add(Conv2D(48,(5, 5),strides=(2, 2),activation='elu'))
    # Layer 4
    model.add(Conv2D(64,(3, 3),activation='elu'))
    # Layer 5
    model.add(Conv2D(64,(3, 3),activation='elu'))

    # Dropout
    model.add(Dropout(0.5))

    # Convert feature maps into one-dimensional vector
    model.add(Flatten())

    # Fully connected layers
    model.add(Dense(100,activation='elu'))

    model.add(Dropout(0.5))
    model.add(Dense(50,activation='elu'))
    model.add(Dropout(0.5))
    model.add(Dense(10,activation='elu'))
    model.add(Dropout(0.5))
    # Final regression output
    model.add(Dense(1))
```

```
    # Compile model
    model.compile(optimizer=Adam(learning_rate=1e-3), loss='mse')
    return model
model = nvidiaModel()
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 31, 98, 24)	1,824
conv2d_1 (Conv2D)	(None, 14, 47, 36)	21,636
conv2d_2 (Conv2D)	(None, 5, 22, 48)	43,248
conv2d_3 (Conv2D)	(None, 3, 20, 64)	27,712
conv2d_4 (Conv2D)	(None, 1, 18, 64)	36,928
dropout (Dropout)	(None, 1, 18, 64)	0
flatten (Flatten)	(None, 1152)	0
dense (Dense)	(None, 100)	115,300
dropout_1 (Dropout)	(None, 100)	0
dense_1 (Dense)	(None, 50)	5,050
dropout_2 (Dropout)	(None, 50)	0
dense_2 (Dense)	(None, 10)	510
dropout_3 (Dropout)	(None, 10)	0
dense_3 (Dense)	(None, 1)	11

Total params: 252,219 (985.23 KB)
Trainable params: 252,219 (985.23 KB)
Non-trainable params: 0 (0.00 B)

WEEK - 4

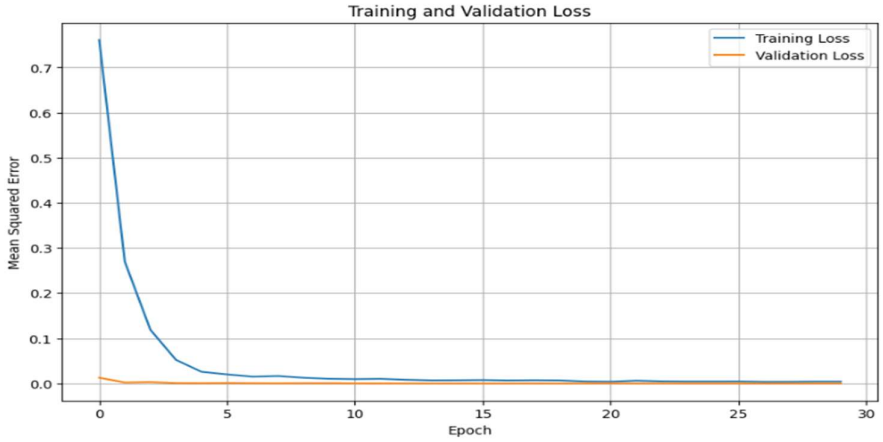
i. Train the model

```
#Calculate training steps
steps_per_epoch = len(x_train) // batch_size
validation_steps = len(x_test) // batch_size

print("Steps per epoch:", steps_per_epoch)
print("Validation steps:", validation_steps)
```

```
Steps per epoch: 12
Validation steps: 3
```

```
#Train the model
history = model.fit(
    train_generator,
    steps_per_epoch=steps_per_epoch,
    validation_data=validation_generator,
    validation_steps=validation_steps,
    epochs=30,verbose=1)
#Plot training and validation loss
plt.figure(figsize=(10, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Mean Squared Error")
plt.legend()
plt.grid(True)
plt.show()
```



j. Test and visualize the output and its extensions

```
#Prepare test images
x_test_processed = np.array([imagePreprocessing(img) for img in x_test],dtype=np.float32)
print("Test data shape:", x_test_processed.shape)
```

Test data shape: (100, 66, 200, 3)

```
#Evaluate the model
test_loss = model.evaluate(x_test_processed, y_test,verbose=1)
print("Test MSE:", test_loss)
```

4/4 ━━━━━━━━━━━━━━━━━━━━ **0s** 33ms/step - loss: 7.6488e-05
Test MSE: 7.648840255569667e-05

```
#Predict steering angles
predictions = model.predict(x_test_processed, verbose=1)
predictions = predictions.flatten()
print("First 10 actual values:")
print(y_test[:10])
print("\nFirst 10 predicted values:")
print(predictions[:10])
```

4/4 ━━━━━━━━━━━━━━━━━━━━ **0s** 57ms/step
First 10 actual values:
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]

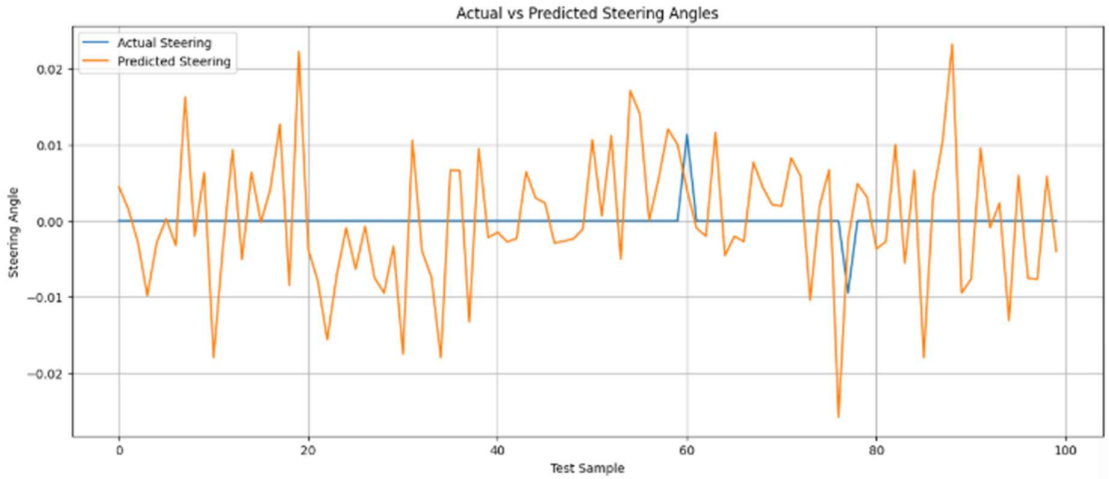
First 10 predicted values:
[0.00449811 0.00152326 -0.00278324 -0.00980588 -0.00280473 0.00026911
-0.00321475 0.01622298 -0.00200712 0.00632703]

```
#Compare actual and predicted values
comparison = pd.DataFrame({'Actual Steering': y_test, 'Predicted Steering': predictions})
comparison.head(10)
```

	Actual Steering	Predicted Steering
0	0.0	0.004498
1	0.0	0.001523
2	0.0	-0.002783
3	0.0	-0.009806
4	0.0	-0.002805
5	0.0	0.000269
6	0.0	-0.003215
7	0.0	0.016223
8	0.0	-0.002007
9	0.0	0.006327

```
#Visualize actual vs predicted steering
plt.figure(figsize=(15, 6))
plt.plot(y_test[:100],label='Actual Steering')
plt.plot(predictions[:100], label='Predicted Steering')

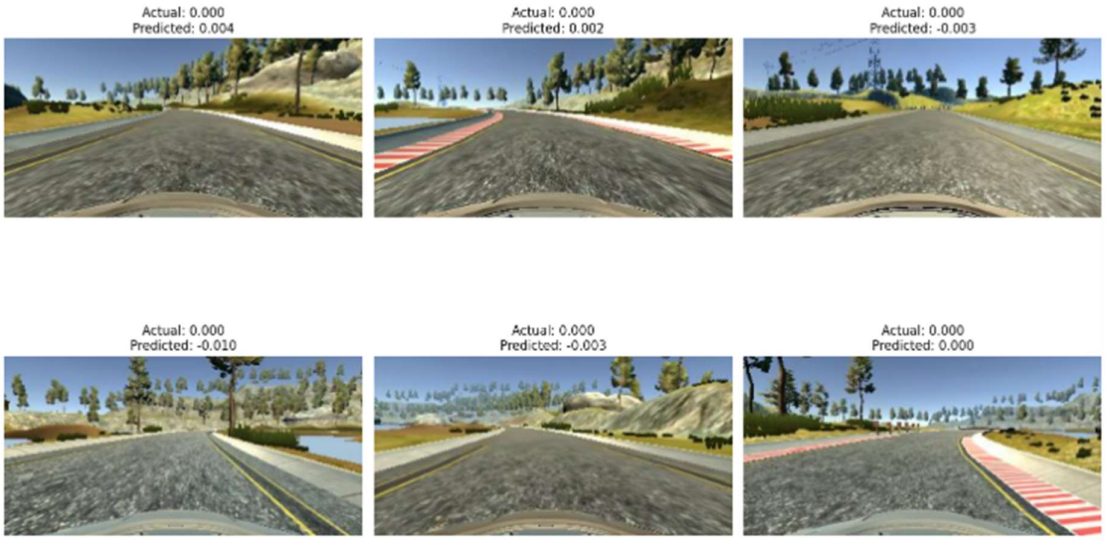
plt.title("Actual vs Predicted Steering Angles")
plt.xlabel("Test Sample")
plt.ylabel("Steering Angle")
plt.legend()
plt.grid(True)
plt.show()
```




```
# Visualize images with predictions
plt.figure(figsize=(15, 10))
for i in range(6):
```

```
    plt.subplot(2, 3, i + 1)
    image = mpimg.imread(x_test[i])
    plt.imshow(image)
    plt.axis('off')
    plt.title(
        f"Actual: {y_test[i]:.3f}\n"
        f"Predicted: {predictions[i]:.3f}"
    )

plt.tight_layout()
plt.show()
```




```
# Calculate baseline vs CNN performance
print("Mean Baseline MSE:", baseline_mse)
print("NVIDIA CNN Test MSE:", test_loss)
```

Mean Baseline MSE: 2.1800227511161654e-06

NVIDIA CNN Test MSE: 7.648840255569667e-05

```
model.save('car.h5')
print("Model saved successfully.")
```

WEEK – 5

Title of the Experiment: Human Activity Recognition (HAR) using Deep Learning

Objective of the experiment: To implement Human Activity Recognition (HAR) using classical Machine Learning and Deep Learning techniques, utilizing a public dataset such as the **UCI HAR dataset**.

a. **Problem definition**

Human Activity Recognition (HAR) is a classification problem where data collected from motion sensors (such as accelerometers and gyroscopes) is used to recognize human activities like walking, sitting, standing, etc. HAR has applications in **health monitoring, sports analysis, and smart home automation.**

Data collected from 30 volunteers performing 6 activities:

- Walking
- Walking Upstairs
- Walking Downstairs
- Sitting
- Standing
- Laying

Features extracted from time-domain & frequency-domain signals.

WEEK - 6

b. **Dataset understanding**

```
# Load the UCI HAR dataset directly from uploaded TXT files

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix,
    precision_score, recall_score, f1_score
)

# File paths: uploaded individually into Colab
X_train_path = "/content/X_train.txt"
y_train_path = "/content/y_train.txt"
X_test_path = "/content/X_test.txt"
y_test_path = "/content/y_test.txt"
features_path = "/content/features.txt"
activities_path = "/content/activity_labels.txt"

# Read feature names
features = pd.read_csv(features_path, sep=r"\s+", header=None)
feature_names = features.iloc[:, 1].tolist()

# Read feature matrices
X_train = pd.read_csv(X_train_path, sep=r"\s+", header=None)
X_test = pd.read_csv(X_test_path, sep=r"\s+", header=None)
```

```
X_train.columns = feature_names
X_test.columns = feature_names

# Read activity labels
y_train = np.loadtxt(y_train_path, dtype=int)
y_test = np.loadtxt(y_test_path, dtype=int)

# Read activity names, preserving names containing spaces
with open(activities_path, "r") as file:
    activity_names = {
        int(line.split(maxsplit=1)[0]):
        line.split(maxsplit=1)[1].strip()
        for line in file if line.strip()
    }

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
print("Number of features:", len(feature_names))
print("Activity names:", activity_names)
```

```
Training data: (7352, 561)
Testing data: (2947, 561)
Number of features: 561
Activity names: {1: 'WALKING', 2: 'WALKING_UPSTAIRS', 3: 'WALKING_DOWNSTAIRS',
4: 'SITTING', 5: 'STANDING', 6: 'LAYING'}
```

```
display(X_train.head())
```

	tBodyAcc-mean()-X	tBodyAcc-mean()-Y	tBodyAcc-mean()-Z	tBodyAcc-std()-X	tBodyAcc-std()-Y	tBodyAcc-std()-Z	tBodyAcc-mad()-X	tBodyAcc-mad()-Y	tBodyAcc-mad()-Z	tBodyAcc-max()-X	...
0	0.288585	-0.020294	-0.132905	-0.995279	-0.983111	-0.913526	-0.995112	-0.983185	-0.923527	-0.934724	...
1	0.278419	-0.016411	-0.123520	-0.998245	-0.975300	-0.960322	-0.998807	-0.974914	-0.957686	-0.943068	...
2	0.279653	-0.019467	-0.113462	-0.995380	-0.967187	-0.978944	-0.996520	-0.963668	-0.977469	-0.938692	...
3	0.279174	-0.026201	-0.123283	-0.996091	-0.983403	-0.990675	-0.997099	-0.982750	-0.989302	-0.938692	...
4	0.276629	-0.016570	-0.115362	-0.998139	-0.980817	-0.990482	-0.998321	-0.979672	-0.990441	-0.942469	...

5 rows × 561 columns

c. **Data cleaning and pre-processing**

```
#  Check data quality
print("Missing values in training data:", X_train.isnull().sum().sum())
print("Missing values in testing data:", X_test.isnull().sum().sum())
print("Duplicate training rows:", X_train.duplicated().sum())

# Check infinite values
print("Infinite values in training data:", np.isinf(X_train.to_numpy()).sum())

# Standardize features
# Fit on training data only to avoid data leakage
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Convert activity IDs from 1–6 to 0–5 for deep learning
y_train_encoded = y_train - 1
y_test_encoded = y_test - 1

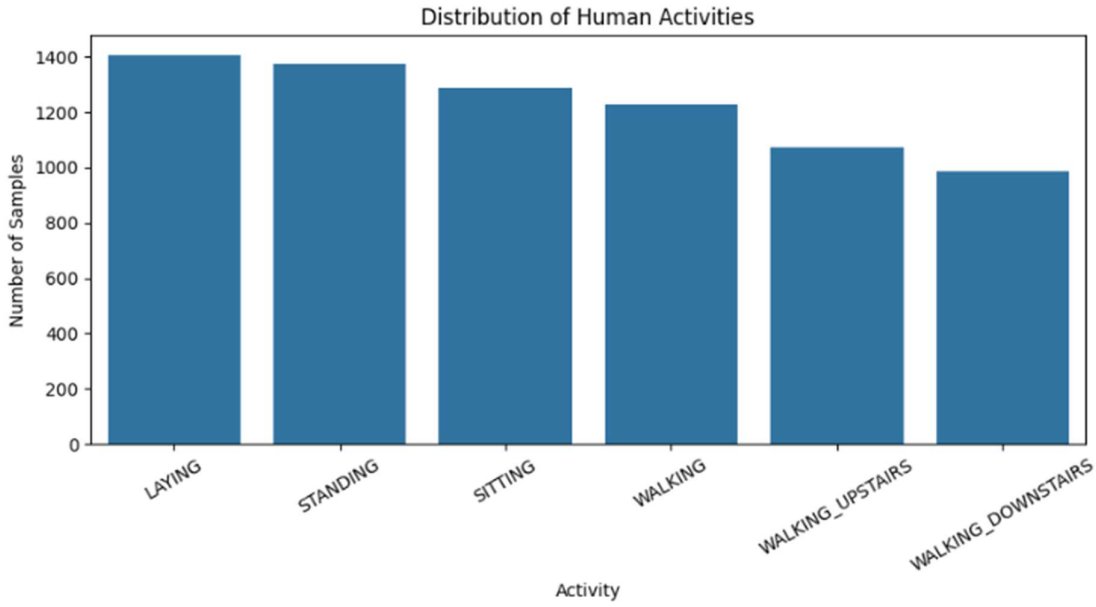
print("Preprocessing completed.")
print("Training features:", X_train_scaled.shape)
print("Testing features:", X_test_scaled.shape)
```

Missing values in training data: 0
Missing values in testing data: 0
Duplicate training rows: 0
Infinite values in training data: 0
Preprocessing completed.
Training features: (7352, 561)
Testing features: (2947, 561)

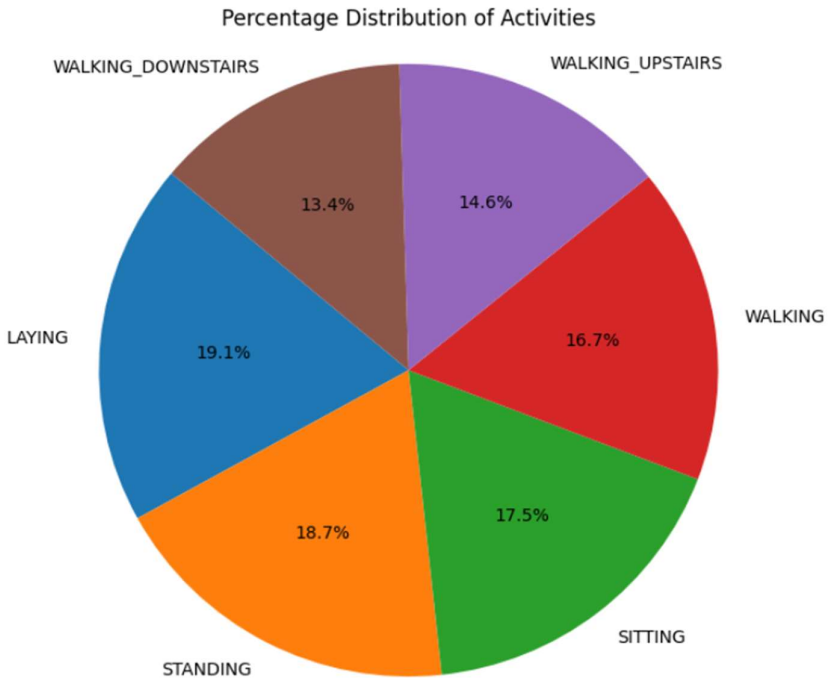
WEEK - 7

d. **EDA univariate analysis**

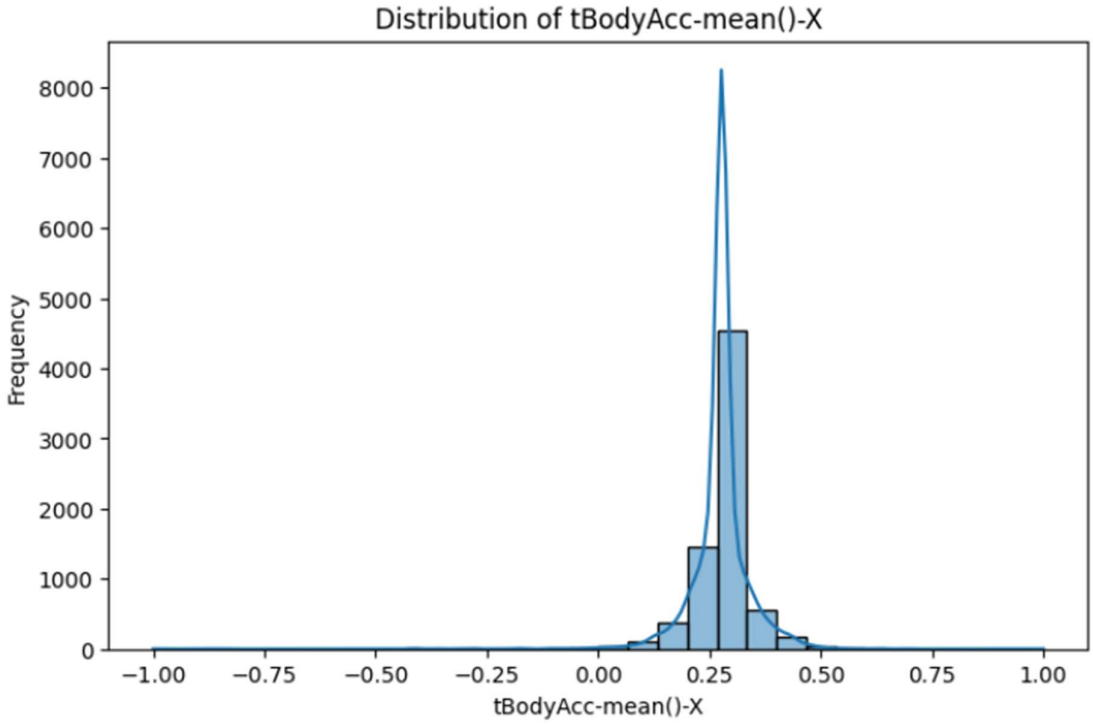
```
# Activity distribution
activity_counts = pd.Series(y_train).map(activity_names).value_counts()
plt.figure(figsize=(9, 5))
sns.barplot(x=activity_counts.index, y=activity_counts.values)
plt.title("Distribution of Human Activities")
plt.xlabel("Activity")
plt.ylabel("Number of Samples")
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()
```




```
plt.figure(figsize=(7, 7))
plt.pie(activity_counts.values, labels=activity_counts.index, autopct="%1.1f%%",
startangle=140)
plt.title("Percentage Distribution of Activities")
plt.axis("equal")
plt.show()
```




```
feature = feature_names[0]
plt.figure(figsize=(8, 5))
sns.histplot(X_train[feature], bins=30, kde=True)
plt.title(f"Distribution of {feature}")
plt.xlabel(feature)
plt.ylabel("Frequency")
plt.show()
```


Distribution of tBodyAcc-mean()-X
Frequency
8000
7000
6000
5000
4000
3000
2000
1000
0
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
tBodyAcc-mean()-X

WEEK - 8

e. **Classical ML models**

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

models = {
    "Logistic Regression": LogisticRegression(max_iter=2000),
    "Random Forest": RandomForestClassifier(
        n_estimators=100, random_state=42, n_jobs=-1
    ),
    "SVM": SVC(kernel="rbf")
}

results = []
for name, model in models.items():
    print("\nTraining:", name)

    model.fit(X_train_scaled, y_train_encoded)
    predictions = model.predict(X_test_scaled)

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test_encoded, predictions),
        "Precision": precision_score(y_test_encoded, predictions, average="weighted"),
        "Recall": recall_score(y_test_encoded, predictions, average="weighted"),
        "F1 Score": f1_score(y_test_encoded, predictions, average="weighted")
    })

    print(classification_report(
        y_test_encoded,
```

```
        predictions,
        labels=np.arange(6),
        target_names=[activity_names[i] for i in range(1, 7)],
        zero_division=0
    ))

results_df = pd.DataFrame(results)
display(results_df.sort_values("Accuracy", ascending=False))
```

```
Training: Logistic Regression
                    precision    recall  f1-score   support

           WALKING       0.94      0.99      0.97       496
  WALKING_UPSTAIRS       0.96      0.95      0.95       471
WALKING_DOWNSTAIRS       0.99      0.94      0.96       420
           SITTING       0.97      0.88      0.92       491
          STANDING       0.89      0.97      0.93       532
            LAYING       1.00      0.99      0.99       537

          accuracy                           0.95      2947
         macro avg       0.96      0.95      0.95      2947
      weighted avg       0.96      0.95      0.95      2947

Training: Random Forest
                    precision    recall  f1-score   support

           WALKING       0.89      0.96      0.92       496
  WALKING_UPSTAIRS       0.89      0.90      0.90       471
WALKING_DOWNSTAIRS       0.97      0.86      0.91       420
           SITTING       0.91      0.89      0.90       491
          STANDING       0.90      0.92      0.91       532
            LAYING       1.00      1.00      1.00       537

          accuracy                           0.93      2947
         macro avg       0.93      0.92      0.92      2947
      weighted avg       0.93      0.93      0.93      2947
```

```
Training: SVM
                    precision    recall  f1-score   support

           WALKING       0.96      0.97      0.97       496
  WALKING_UPSTAIRS       0.93      0.97      0.95       471
WALKING_DOWNSTAIRS       0.98      0.92      0.95       420
           SITTING       0.94      0.90      0.92       491
          STANDING       0.92      0.95      0.93       532
            LAYING       1.00      1.00      1.00       537

          accuracy                           0.95      2947
         macro avg       0.95      0.95      0.95      2947
      weighted avg       0.95      0.95      0.95      2947
```

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.954869	0.956650	0.954869	0.954809
2	SVM	0.952155	0.952594	0.952155	0.952068
1	Random Forest	0.926026	0.927377	0.926026	0.925957

f. **Deep learning models**

```
# Prepare data for deep learning
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Conv1D, MaxPooling1D
from tensorflow.keras.layers import Flatten, Dense, Dropout, LSTM
from tensorflow.keras.callbacks import EarlyStopping

# Split only the training data to create a validation set
X_tr, X_val, y_tr, y_val = train_test_split(
```

```
    X_train_scaled,
    y_train_encoded,
    test_size=0.1,
    random_state=42,
    stratify=y_train_encoded
)

# CNN and LSTM expect three-dimensional input
X_tr_dl = X_tr.astype("float32").reshape(X_tr.shape[0], X_tr.shape[1], 1)
X_val_dl = X_val.astype("float32").reshape(X_val.shape[0], X_val.shape[1], 1)
X_test_dl = X_test_scaled.astype("float32").reshape(
    X_test_scaled.shape[0], X_test_scaled.shape[1], 1
)

print("Training:", X_tr_dl.shape)
print("Validation:", X_val_dl.shape)
print("Testing:", X_test_dl.shape)

Training: (6616, 561, 1)
Validation: (736, 561, 1)
Testing: (2947, 561, 1)

# 1D CNN
cnn_model = Sequential([
    Input(shape=(561, 1)),
    Conv1D(64, 3, activation="relu"),
    MaxPooling1D(2),
    Dropout(0.3),
    Conv1D(128, 3, activation="relu"),
    MaxPooling1D(2),
    Flatten(),
```

```
    Dense(128, activation="relu"),
    Dropout(0.3),
    Dense(6, activation="softmax")
])

cnn_model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
metrics=["accuracy"])

early_stop = EarlyStopping(monitor="val_loss", patience=3, restore_best_weights=True)

cnn_history = cnn_model.fit(
    X_tr_dl, y_tr,
    validation_data=(X_val_dl, y_val),
    epochs=15,
    batch_size=64,
    callbacks=[early_stop],
    verbose=1
)

cnn_loss, cnn_accuracy = cnn_model.evaluate(X_test_dl, y_test_encoded)
print("CNN Test Accuracy:", round(cnn_accuracy, 4))

CNN Test Accuracy: 0.9467

# LSTM
lstm_model = Sequential([
    Input(shape=(561, 1)),
    LSTM(64),
    Dropout(0.3),
    Dense(64, activation="relu"),
    Dense(6, activation="softmax")
```

```
])

lstm_model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
metrics=["accuracy"])

lstm_history = lstm_model.fit(
    X_tr_dl, y_tr,
    validation_data=(X_val_dl, y_val),
    epochs=15,
    batch_size=64,
    callbacks=[early_stop],
    verbose=1
)

lstm_loss, lstm_accuracy = lstm_model.evaluate(X_test_dl, y_test_encoded)
print("LSTM Test Accuracy:", round(lstm_accuracy, 4))
Epoch 1/15
104/104 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 19s 173ms/step - accuracy: 0.4589 -
loss: 1.2209 - val_accuracy: 0.6399 - val_loss: 0.8397
Epoch 2/15
104/104 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 20s 169ms/step - accuracy: 0.6326 -
loss: 0.7971 - val_accuracy: 0.6698 - val_loss: 0.7324
Epoch 3/15
104/104 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 20s 162ms/step - accuracy: 0.6778 -
loss: 0.7114 - val_accuracy: 0.7160 - val_loss: 0.6078
93/93 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6s 64ms/step - accuracy: 0.6305 - loss:
0.8926
LSTM Test Accuracy: 0.6305

# Compare deep learning models
cnn_predictions = np.argmax(cnn_model.predict(X_test_dl, verbose=0), axis=1)
```

```
lstm_predictions = np.argmax(lstm_model.predict(X_test_dl, verbose=0), axis=1)
deep_results = pd.DataFrame([
    {
        "Model": "CNN",
        "Accuracy": accuracy_score(y_test_encoded, cnn_predictions)
    },
    {
        "Model": "LSTM",
        "Accuracy": accuracy_score(y_test_encoded, lstm_predictions)
    }
])

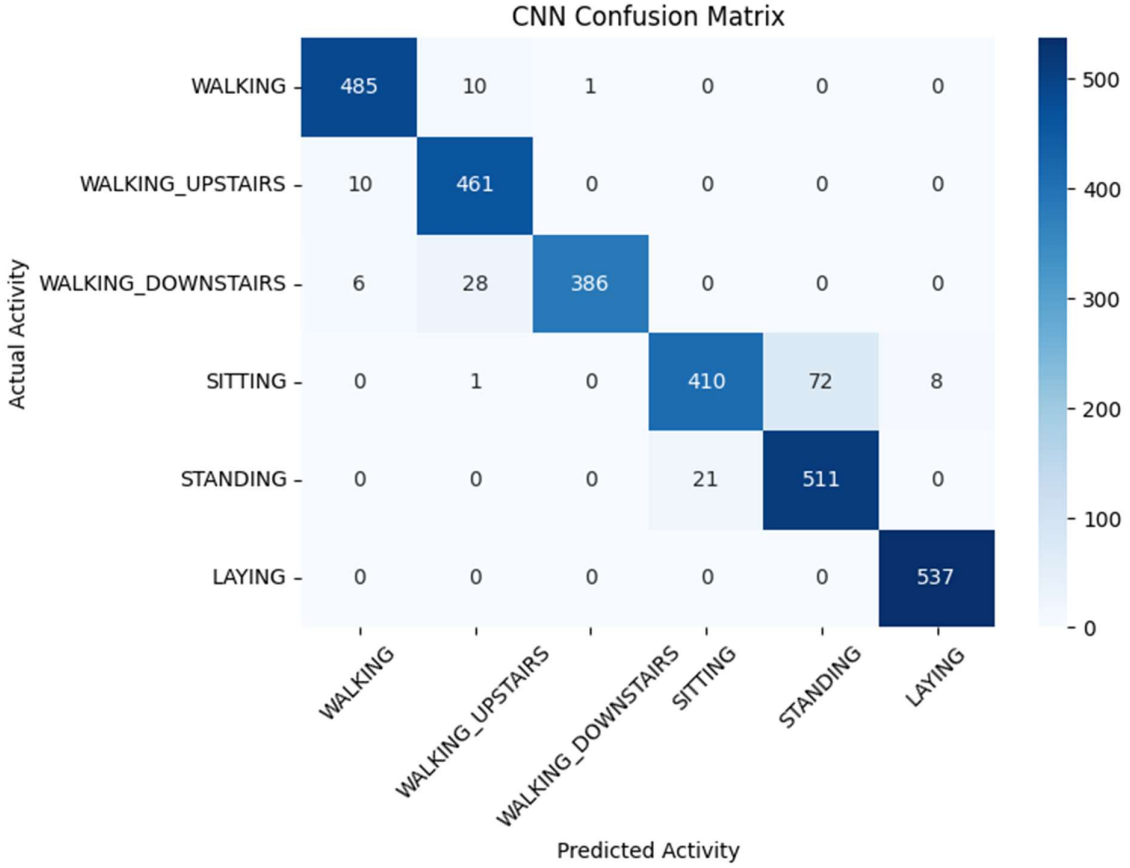
display(deep_results)

# CNN confusion matrix
cm = confusion_matrix(y_test_encoded, cnn_predictions)

plt.figure(figsize=(8, 6))
sns.heatmap(
    cm, annot=True, fmt="d", cmap="Blues",
    xticklabels=[activity_names[i] for i in range(1, 7)],
    yticklabels=[activity_names[i] for i in range(1, 7)]
)

plt.title("CNN Confusion Matrix")
plt.xlabel("Predicted Activity")
plt.ylabel("Actual Activity")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

	Model	Accuracy
0	CNN	0.946725
1	LSTM	0.630472



WEEK – 9

Title of the Experiment: Quora question pair similarity problem

Objective of the Experiment:

Understand the importance of duplicate question detection in NLP applications. Perform Exploratory Data Analysis (EDA) on the Quora Question Pairs dataset. Extract key text-based features (TF-IDF, Word2Vec, Sentence Embeddings). Implement traditional ML models (Logistic Regression, SVM, Random Forest).Build Deep Learning models (LSTMs, Bi-LSTMs, BERT, etc.) for text similarity.). Implemented performance using precision, recall, F1-score, and accuracy.

a. **Business real world problem definition**

The objective is to identify whether two questions posted on a question-answering platform express the same intent. Duplicate detection can help reduce repeated questions, improve search, and direct users to existing answers.

b. **Business objectives and constraints**

- Detect duplicate question pairs accurately.
- Reduce unnecessary repeated questions.
- Minimize false positives (different questions incorrectly marked as duplicates) and false negatives (duplicates missed).
- Provide duplicate probabilities when possible, allowing the decision threshold to be adjusted.
- Handle large volumes of text within reasonable memory and processing time.

c. **Mapping to an ML problem data overview**

```
# Load and inspect the dataset
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("train.csv")
```

```
print("Dataset shape:", df.shape)
print("\nColumns:", df.columns.tolist())
print("\nMissing values:\n", df[["question1", "question2", "is_duplicate"]].isnull().sum())

display(df.head())
print("\nTarget distribution:")
print(df["is_duplicate"].value_counts())
```

```
Dataset shape: (404290, 6)
Columns: ['id', 'qid1', 'qid2', 'question1', 'question2', 'is_duplicate']
Missing values:
 question1       1
question2       2
is_duplicate    0
dtype: int64
```

	id	qid1	qid2	question1	question2	is_duplicate
0	0	1	2	What is the step by step guide to invest in sh...	What is the step by step guide to invest in sh...	0
1	1	3	4	What is the story of Kohinoor (Koh-i-Noor) Dia...	What would happen if the Indian government sto...	0
2	2	5	6	How can I increase the speed of my internet co...	How can Internet speed be increased by hacking...	0
3	3	7	8	Why am I mentally very lonely? How can I solve...	Find the remainder when [math]23^{24} [/math] i...	0

```
Target distribution:
is_duplicate
0    255027
1    149263
Name: count, dtype: int64
```

WEEK - 10

d. Mapping to an ML problem and performance measures

- Accuracy: proportion of correct predictions.
- Precision: how many predicted duplicate pairs are actually duplicates.
- Recall: how many actual duplicate pairs are detected.
- F1-score: harmonic mean of precision and recall.
- Log-loss: evaluates predicted probabilities and penalizes confident incorrect predictions. It is the Kaggle competition's evaluation metric.

e. Mapping to an ML problem train-test split

```
# Clean essential fields and split labeled data
from sklearn.model_selection import train_test_split

df = df.dropna(subset=["is_duplicate"]).copy()
df["question1"] = df["question1"].fillna("")
df["question2"] = df["question2"].fillna("")
df["is_duplicate"] = df["is_duplicate"].astype(int)
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42, stratify=df["is_duplicate"])

print("Training shape:", train_df.shape)
print("Testing shape:", test_df.shape)
print("\nTraining label distribution:")
print(train_df["is_duplicate"].value_counts(normalize=True))
```

```
Training shape: (323432, 6)
Testing shape: (80858, 6)
Training label distribution:
is_duplicate
0    0.630803
1    0.369197
```

Name: proportion, dtype: float64

f. EDA basic statistics, feature extraction

```
# Basic statistics
print("Duplicate ratio:", df["is_duplicate"].mean())
print("\nDataset statistics:")
print(df[["is_duplicate"]].describe())

df["q1_length"] = df["question1"].str.len()
df["q2_length"] = df["question2"].str.len()
df["length_difference"] = (df["q1_length"] - df["q2_length"]).abs()
df["q1_word_count"] = df["question1"].str.split().str.len()
df["q2_word_count"] = df["question2"].str.split().str.len()

display(df[["question1", "question2", "is_duplicate","q1_length", "q2_length",
"length_difference"]].head())
```

```
Duplicate ratio: 0.369197853026293
Dataset statistics:
       is_duplicate
count  404290.000000
mean        0.369198
std         0.482588
min         0.000000
25%         0.000000
50%         0.000000
75%         1.000000
max         1.000000
```

VNRVJIET

NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:

	question1	question2	is_duplicate	q1_length	q2_length	length_difference
0	What is the step by step guide to invest in sh...	What is the step by step guide to invest in sh...	0	66	57	9
1	What is the story of Kohinoor (Koh-i-Noor) Dia...	What would happen if the Indian government sto...	0	51	88	37
2	How can I increase the speed of my internet co...	How can Internet speed be increased by hacking...	0	73	59	14
3	Why am I mentally very lonely? How can I solve...	Find the remainder when [math]23^{24}[/math] i...	0	50	65	15
4	Which one dissolve in water quikly sugar, salt...	Which fish would survive in salt water?	0	76	39	37

WEEK - 11

g. EDA text processing, advanced feature extraction

```
import re

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"[^a-z0-9\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

def pair_features(q1, q2):
    words1 = set(clean_text(q1).split())
    words2 = set(clean_text(q2).split())

    common = len(words1 & words2)
    total = len(words1 | words2)

    return pd.Series({
        "q1_length": len(str(q1)),
        "q2_length": len(str(q2)),
        "length_difference": abs(len(str(q1)) - len(str(q2))),
        "common_words": common,
        "jaccard_similarity": common / total if total else 0
    })

train_df = train_df.copy()
test_df = test_df.copy()
train_df["question1_clean"] = train_df["question1"].apply(clean_text)
train_df["question2_clean"] = train_df["question2"].apply(clean_text)
test_df["question1_clean"] = test_df["question1"].apply(clean_text)
test_df["question2_clean"] = test_df["question2"].apply(clean_text)
```

```
display(train_df[["question1_clean", "question2_clean", "is_duplicate"]].head())

train_pair_features = pd.DataFrame([
    pair_features(q1, q2)
    for q1, q2 in zip(train_df["question1"], train_df["question2"])
])

test_pair_features = pd.DataFrame([
    pair_features(q1, q2)
    for q1, q2 in zip(test_df["question1"], test_df["question2"])
])

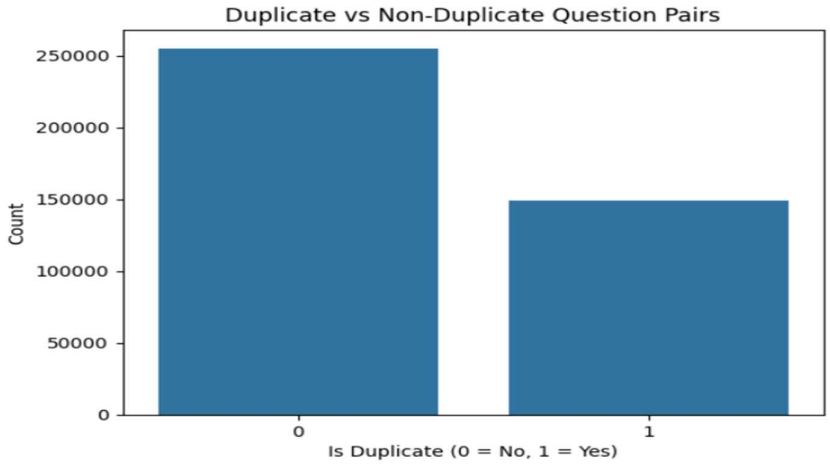
display(train_pair_features.head())
```

	question1_clean	question2_clean	is_duplicate
341317	what is an anagram	is there an anagram for anagram	0
237560	social media what does it mean when a guy dele...	my boyfriend treasures memories he had with hi...	0
280719	is this a good time to buy apple stock	is it a good time to buy apple stock at 92	0
288940	what are the consequences of getting high on h...	what is the number of lok sabha constituencies...	0
27093	what are some good mixes with watermelon vodka	what s a good mixer for watermelon vodka	1

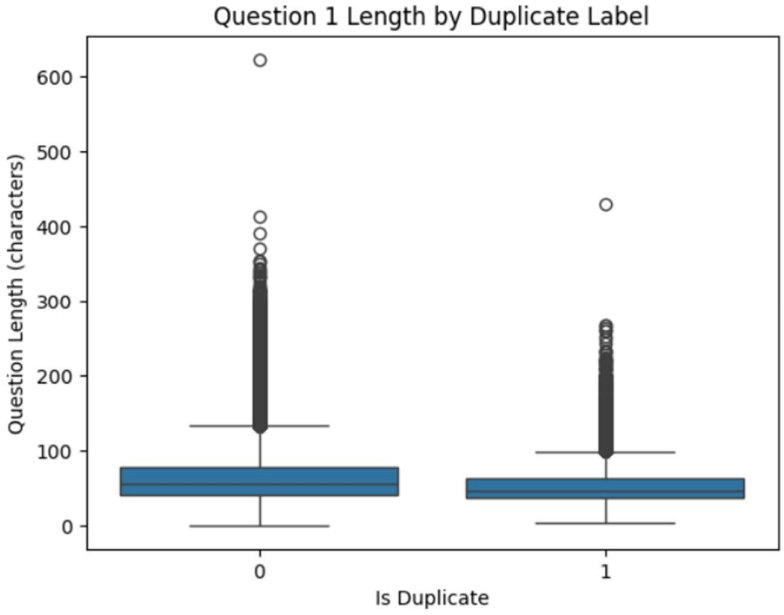
	q1_length	q2_length	length_difference	common_words	jaccard_similarity
0	19.0	32.0	13.0	3.0	0.500000
1	112.0	144.0	32.0	7.0	0.184211
2	39.0	44.0	5.0	8.0	0.666667
3	52.0	93.0	41.0	3.0	0.142857
4	47.0	41.0	6.0	4.0	0.333333

h. EDA feature analysis, data visualization

```
sns.countplot(data=df, x="is_duplicate")
plt.title("Duplicate vs Non-Duplicate Question Pairs")
plt.xlabel("Is Duplicate (0 = No, 1 = Yes)")
plt.ylabel("Count")
plt.show()
```




```
# Compare question lengths by class
sns.boxplot(data=df, x="is_duplicate",y="q1_length")
plt.title("Question 1 Length by Duplicate Label")
plt.xlabel("Is Duplicate")
plt.ylabel("Question Length (characters)")
plt.show()
```



WEEK - 12

i. EDA IT-IDF weighting word2vec factorization

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(max_features=50000, ngram_range=(1, 2), min_df=2, sublinear_tf=True)
train_questions = pd.concat([train_df["question1_clean"], train_df["question2_clean"]])
vectorizer.fit(train_questions)

q1_train_tfidf = vectorizer.transform(train_df["question1_clean"])
q2_train_tfidf = vectorizer.transform(train_df["question2_clean"])

q1_test_tfidf = vectorizer.transform(test_df["question1_clean"])
q2_test_tfidf = vectorizer.transform(test_df["question2_clean"])

print("TF-IDF shape:", q1_train_tfidf.shape)

TF-IDF shape: (323432, 50000)

# Combine question-pair features
from scipy.sparse import hstack

X_train_tfidf = hstack([
    abs(q1_train_tfidf - q2_train_tfidf),
    q1_train_tfidf.multiply(q2_train_tfidf),
    train_pair_features[[
        "length_difference", "common_words", "jaccard_similarity"
    ]].to_numpy()
]).tocsr()

X_test_tfidf = hstack([
```

```
    abs(q1_test_tfidf - q2_test_tfidf),
    q1_test_tfidf.multiply(q2_test_tfidf),
    test_pair_features[[
        "length_difference", "common_words", "jaccard_similarity"
    ]].to_numpy()
]).tocsr()

y_train = train_df["is_duplicate"].to_numpy()
y_test = test_df["is_duplicate"].to_numpy()

print("Training feature matrix:", X_train_tfidf.shape)
print("Testing feature matrix:", X_test_tfidf.shape)
```

```
Training feature matrix: (323432, 100003)
Testing feature matrix: (80858, 100003)
```

j. ML models loading, random model

```
from sklearn.metrics import (accuracy_score, precision_score, recall_score,f1_score, log_loss)

def evaluate_model(y_true, y_pred, y_prob=None):
    print("Accuracy:", round(accuracy_score(y_true, y_pred), 4))
    print("Precision:", round(precision_score(y_true, y_pred, zero_division=0), 4))
    print("Recall:", round(recall_score(y_true, y_pred, zero_division=0), 4))
    print("F1 Score:", round(f1_score(y_true, y_pred, zero_division=0), 4))

    if y_prob is not None:
        print("Log Loss:", round(log_loss(y_true, y_prob), 4))

# Predict random class labels
rng = np.random.default_rng(42)
random_predictions = rng.integers(0, 2, size=len(y_test))
```

```
print("Random Model Results:")
evaluate_model(y_test, random_predictions)
```

```
Random Model Results:
Accuracy: 0.4996
Precision: 0.369
Recall: 0.5004
F1 Score: 0.4248
```

k. ML models logistics regressions and linear SVM

```
# Logistic Regression
from sklearn.linear_model import LogisticRegression

lr_model = LogisticRegression(max_iter=1000, solver="liblinear")
lr_model.fit(X_train_tfidf, y_train)
lr_predictions = lr_model.predict(X_test_tfidf)
lr_probabilities = lr_model.predict_proba(X_test_tfidf)[:, 1]

print("Logistic Regression Results:")
evaluate_model(y_test, lr_predictions, lr_probabilities)
```

```
Logistic Regression Results:
Accuracy: 0.8179
Precision: 0.7754
Recall: 0.7136
F1 Score: 0.7432
Log Loss: 0.3934
```

```
# Linear SVM
from sklearn.svm import LinearSVC
```

```
svm_model = LinearSVC(random_state=42)
svm_model.fit(X_train_tfidf, y_train)
svm_predictions = svm_model.predict(X_test_tfidf)

print("Linear SVM Results:")
evaluate_model(y_test, svm_predictions)
```

```
Linear SVM Results:
Accuracy: 0.828
Precision: 0.771
Recall: 0.7596
F1 Score: 0.7653
```

VNRVJIET

NAME OF LAB: _____ WEEK: _____

EXPERIMENT NAME:

WEEK – 13

Title of the Experiment: Music generation using deep learning

Objective of the Experiment:

- Understand how deep learning models can generate music.
- Explore Char-RNN and its role in music generation using ABC notation.
- Preprocess ABC music notation datasets for training.
- Build a Recurrent Neural Network (RNN) model for music sequence learning.
- Implement MIDI-based music generation using deep learning.
- Generate unique tabla music sequences using AI-based models

a. Real-world problem and music representation

Music generation using deep learning trains a model to learn patterns in existing music and generate new sequences. Applications include music composition, game soundtracks, musical assistance, and experimenting with different styles.

- **ABC notation:** represents music as readable text using characters and symbols for notes, rhythm, meter, and key.
- **MIDI:** represents musical events such as note pitch, note-on/note-off timing, velocity, and instrument information. It is not normally a recording of the actual sound.
- **Objective:** train a sequence model to predict the next character or musical event from previous ones.

b. Char-RNN with abc –notation char CNN model

A Character-Level Recurrent Neural Network (Char-RNN) learns patterns in ABC notation one character at a time. An LSTM layer learns dependencies across characters, and the output layer predicts the next character.

For example, C D E G represents a sequence of notes. The model learns which characters and note patterns commonly follow one another.

WEEK - 14

c. Char-RNN with abc –notation data preparation

```
music_data = """X:1
T:Simple Rhythm
M:4/4
L:1/8
K:C
C D E G | G E D C |
C C G G | A A G2 |
"""
with open("/content/music.abc", "w", encoding="utf-8") as file:
    file.write(music_data)

print("ABC file created successfully!")
print(music_data)
```

```
ABC file created successfully!
X:1
T:Simple Rhythm
M:4/4
L:1/8
K:C
C D E G | G E D C |
C C G G | A A G2 |
```

```
import re
import numpy as np

# Remove tune-number metadata, but retain musical headers
cleaned_data = re.sub(r"(?m)^X:.*\n?", "", music_data).strip()
```

```
# Create character-to-number mappings
chars = sorted(set(cleaned_data))
char_to_num = {char: i for i, char in enumerate(chars)}
num_to_char = {i: char for char, i in char_to_num.items()}

sequences = np.array([char_to_num[c] for c in cleaned_data])

vocab_size = len(chars)
print("Cleaned ABC text:\n", cleaned_data)
print("Vocabulary size:", vocab_size)
print("Encoded sequence shape:", sequences.shape)
```

```
Cleaned ABC text:
 T:Simple Rhythm
M:4/4
L:1/8
K:C
C D E G | G E D C |
C C G G | A A G2 |
Vocabulary size: 28
Encoded sequence shape: (70,)
```

Neural networks process numerical inputs, so each unique character is assigned an integer.

d. Char-RNN with abc –notation many to many RNN time distribution

```
#Create input sequences of 40 characters and their next-character targets:
SEQ_LEN = 40
X, y = [], []

for i in range(len(sequences) - SEQ_LEN):
    X.append(sequences[i:i + SEQ_LEN])
```

```
    y.append(sequences[i + 1:i + SEQ_LEN + 1])

X = np.array(X)
y = np.array(y)

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (30, 40)
y shape: (30, 40)
```

e. Char-RNN with abc –notation state full RNN,model architecture

The embedding layer represents characters as vectors; the LSTM learns sequence patterns; and the time-distributed dense layer predicts character probabilities.

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, TimeDistributed

model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=32),
    LSTM(64, return_sequences=True),
    TimeDistributed(Dense(vocab_size, activation="softmax"))
])

model.compile(optimizer="adam", loss="sparse_categorical_crossentropy")
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
time_distributed (TimeDistributed)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)

model.fit(X,y,epochs=30,batch_size=32,verbose=1)

Epoch 1/30

1/1 ━━━━━━━━━━━━━━━━━━━━ **10s** 10s/step - loss: 3.3331

Epoch 2/30

1/1 ━━━━━━━━━━━━━━━━━━━━ **0s** 142ms/step - loss: 3.3177

f. Char-RNN with abc –notation music generation, generate tabla music.

After training, provide a seed sequence to the model. It predicts the next character repeatedly to extend the music.

```
def generate_music(seed, length=200):
    output = seed
    for _ in range(length):
        # Use the most recent SEQ_LEN characters
        context = output[-SEQ_LEN:]
        encoded = np.array([[char_to_num[c] for c in context]])
        probabilities = model.predict(encoded, verbose=0)[0, -1]

        # Sample from the predicted character probabilities
        probabilities = probabilities / probabilities.sum()
        next_id = np.random.choice(vocab_size, p=probabilities)

        output += num_to_char[next_id]
    return output
```

```
seed_text = cleaned_data[-SEQ_LEN:]
generated_music = generate_music(seed_text, length=200)
print(generated_music)

with open("/content/generated_music.abc", "w", encoding="utf-8") as file:
    file.write(generated_music)

print("Saved: generated_music.abc")
```

```
C
C D E G | G E D C |
C C G G | A A G2 |CCCE|2DC
|DL
 DCGGG C1A |

K GD4CE GEG C hGC|    G G: DK C|GGDG G  EGG   CRC|lGKG   C
  G
 ME
8pKC|EC  C   D:D   | G:DED   CeGGC

CGG|CGE8  G
DGC
GE   CC  4G |K / GiD D

 A C  D4 DDAy  C
G
Saved: generated_music.abc
```

g. MIDI music generation.

```
#Read MIDI notes and prepare sequences
import glob
```

```
import pretty_midi
import numpy as np

midi_files = glob.glob("/content/*.mid")
print("MIDI files found:", len(midi_files))

all_pitches = []
for path in midi_files:
    try:
        midi_data = pretty_midi.PrettyMIDI(path)
        for instrument in midi_data.instruments:
            for note in instrument.notes:
                all_pitches.append((note.start, note.pitch))

    except Exception as e:
        print("Skipping unreadable file:", path)

# Sort notes by their start time
all_pitches.sort(key=lambda item: item[0])
pitches = np.array([pitch for _, pitch in all_pitches], dtype=np.int32)

print("Total notes:", len(pitches))
print("First pitches:", pitches[:20])

MIDI files found: 2
Total notes: 7961
First pitches: [59 56 52 40 59 56 52 42 35 38 49 42 42 42 42 42 59 57 52 42]

#Train an LSTM to predict the next MIDI pitch
import tensorflow as tf
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Embedding, LSTM, Dense

SEQ_LEN = 32

if len(pitches) <= SEQ_LEN:
    raise ValueError("Upload more MIDI files containing enough notes.")

X, y = [], []
for i in range(len(pitches) - SEQ_LEN):
    X.append(pitches[i:i + SEQ_LEN])
    y.append(pitches[i + SEQ_LEN])

X = np.array(X)
y = np.array(y)

model_midi = Sequential([Embedding(input_dim=128, output_dim=32), LSTM(64),
Dense(128, activation="softmax")])
model_midi.compile(optimizer="adam", loss="sparse_categorical_crossentropy")
model_midi.fit(X, y, epochs=10, batch_size=32)

Epoch 1/10
248/248 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 13s 34ms/step - loss: 3.1156

#Generate and save a MIDI file
seed = list(pitches[:SEQ_LEN])
generated_pitches = seed.copy()

for _ in range(100):
    context = np.array([generated_pitches[-SEQ_LEN:]])
    probabilities = model_midi.predict(context, verbose=0)[0]
    next_pitch = np.random.choice(128, p=probabilities / probabilities.sum())
```

```
    generated_pitches.append(int(next_pitch))

# Create a basic MIDI instrument track
output = pretty_midi.PrettyMIDI()
piano = pretty_midi.Instrument(program=0)

for i, pitch in enumerate(generated_pitches[SEQ_LEN:]):
    start = i * 0.3
    end = start + 0.25
    piano.notes.append(pretty_midi.Note(velocity=90,pitch=int(pitch),start=start,end=end))

output.instruments.append(piano)
output.write("/content/generated_midi.mid")
print("Generated MIDI saved successfully.")

Generated MIDI saved successfully.
```